SİSTEM MİMARİSİ & YOL HARİTASI DOKÜMANI

HarmonyBlend (HB)

B2B Sipariş & Bayi Yönetim Sistemi — Sistem Mimarisi, Veri Modeli ve Fazlandırılmış Yol Haritası

Oluşturulma tarihi: 23 Ağustos 2026
docs/roadmap.md kaynağından otomatik üretildi

HarmonyBlend (HB)

Fabrika/genel merkez ile franchise bayiler arasındaki sipariş, ödeme ve cari tahsilat süreçlerini yöneten çok markalı bayi portalı. Bu doküman mimari, veri modeli, iş akışı, teknoloji ve ekran planını içerir.

- **Kapsam** — Tunam çatısı altında çoklu alt marka (Musluoğlu, Dilim Börek)
- **Veritabanı** — Neon Postgres, ilk günden sabit
- **Ödeme** — Cari · Nakit/Kapıda Ödeme · Sanal POS (mock)

---

1. Fazlandırılmış Yol Haritası

Yedi faz; her biri bir öncekinin üstüne çalışır durumda kalacak şekilde kurgulandı. Sanal POS entegrasyonu bilerek en sona bırakıldı — mock sağlayıcı arayüzü Faz 3'te kurulduğu için Faz 6 bir "değiştirme" işlemi, yeniden yazma değil. Aynı gerekçeyle bir ERP entegrasyonu ihtimali de en sona (Faz 7) bırakıldı; Faz 4'teki sipariş düzenleme/silme özelliği bu ihtimali gözeterek tasarlanacak (bkz. Faz 4 ve Faz 7 notları) ki ileride yeniden tasarlanmasın.

Faz 1 — Temel Altyapı & Veri Modeli Kurulumu ✅ Tamamlandı

Hedef: Neon DB bağlantısı, ORM ve auth iskeleti — projenin geri kalanının üstüne oturacağı zemin.

- Neon Postgres projesi + connection string'in ortam değişkenlerine tanımlanması (bir kere, kalıcı)
- Drizzle ORM kurulumu ve migration pipeline'ı
- `companies`, `brands`, `users`, `dealers`, `dealer_brands` ilk migration
- Rol bazlı auth iskeleti: firma admini, marka yöneticisi, bayi kullanıcısı

Çıkış kriteri — boş fakat çalışan bir admin girişi; veritabanına yazma/okuma uçtan uca doğrulanmış.

Faz 2 — Marka, Bayi ve Ürün Yönetimi ✅ Tamamlandı

Hedef: Admin panelin çekirdek CRUD ekranları; marka bazlı ayrışmanın veri katmanında kurulması.

- Alt marka CRUD'u ve marka bazlı yetkilendirme
- Bayi CRUD'u + `dealer_brands` ataması
- Kategori/ürün CRUD'u — marka bazlı stok ve ambalaj alanlarıyla
- ~~`payment_methods` tanımları ve bayi/marka bazlı aktivasyon~~ — bu yaklaşım daha sonra tamamen kaldırıldı, bkz. Faz 3 sonundaki not

Çıkış kriteri — bir admin yeni bir bayiyi bir markaya bağlayıp, o bayinin sadece o markanın kataloğunu gördüğünü doğrulayabiliyor.

Faz 3 — Sipariş & Mock Ödeme Akışı ✅ Tamamlandı

Hedef: Bayi tarafında sepetten siparişe; ödeme tipine göre dallanan checkout ve durum makinesi.

- Bayi kullanıcı hesabı oluşturma ekranı: admin bir bayiye kullanıcı adı/e-posta + şifre tanımlayıp `dealer_user` rolüyle giriş yapabilecek bir hesap açıyor (`/admin/dealers/[id]`) — önceden yalnızca elle (seed script) yapılabiliyordu.
- Sepet ve checkout ekranı (marka bazlı katalog)
- Ödeme tipine göre dinamik checkout adımları
- `orders` / `order_items` oluşturma, durum makinesi implementasyonu
- `PaymentProvider` arayüzü + `MockPosProvider`
- `order_status_history` ile denetim izi
- Toast bildirim sistemi (sipariş oluşturuldu / ödeme başarısız gibi başarı-hata mesajları için — `alert()` / `confirm()` yerine)
- Erişilebilirlik dokunuşları (`focus-visible` odak halkası, `prefers-reduced-motion` desteği)
- Sidebar menüsünü açılır/kapanır (daraltılabilir) yapma — admin ve bayi panelinde, tercih `localStorage` 'da kalıcı
- Sidebar'ı sticky yapma — sayfa aşağı kaydırılsa bile kenar çubuğundaki öğeler sabit kalıyor, içerikle birlikte yukarı/aşağı kaymıyor
- Bayi listesinde "Düzenle" butonu — düzenleme artık ayrı bir sayfada değil, "Ekle" ile aynı 3 sekmeli modal üzerinden mevcut bayinin ayarları aynı yerden değiştirilebiliyor
- "Ekle/Düzenle" modalındaki maksimum bakiye limiti alanı para birimi formatlı (`CurrencyInput`) — düz sayı yerine otomatik binlik ayraç ve ₺ sembolüyle (`200.000₺`) gösteriliyor
- Bayi listesinde marka/durum filtre formu (`dealer-filter-form.tsx`)
- Admin → bayi kullanıcısı olarak giriş (impersonation): admin, bir bayinin panelini o bayi kullanıcısı kimliğiyle görüntüleyebiliyor (checkout/sipariş akışlarını bayi gözünden test etmek için). Kimlik değişimi NextAuth JWT `update` trigger'ı ile sunucu tarafında doğrulanıyor (`src/auth.ts`); bayi panelinde "orijinal hesaba dön" butonu ile geri dönülüyor. Roadmap'te öngörülmemiş, iş ihtiyacından doğan bir ek kapsamdı.
- Ödeme yöntemi aktivasyon sistemi (`payment_methods` / `dealer_payment_methods` tabloları, bayi/marka bazlı manuel açma-kapama) Faz 3 sonrasında tamamen kaldırıldı — Payna Grup Portal'ın aksine HarmonyBlend'in ERP entegrasyonu olmadığından bu ayrım gereksizdi. Artık her bayi cari/nakit/sanal POS'un hepsini kullanabiliyor; cari yalnızca mevcut bakiye + sipariş tutarı `maxBalanceLimit` 'i aşarsa otomatik kapanıyor. `orders.payment_method_id` (FK) yerine sabit bir enum sütun (`orders.payment_method`) kullanılıyor.
- Toast bildirim sistemi admin paneline de genişletildi — marka/bayi/ ürün/kategori/kullanıcı CRUD işlemleri ve sipariş durum geçişleri artık başarı/hata toast'ı gösteriyor (önceden yalnızca bayi tarafında vardı). Yeniden kullanılabilir `ToastForm` bileşeni (`src/components/ toast-form.tsx`) ile server action'lar client tarafında çağrılıyor.
- Sipariş silme (admin) ve ürün/marka için "zorla sil" — normal silme, sipariş geçmişinde kullanılan ürün/markayı `order_items` FK'sinin engellemesiyle reddediyor (bkz. üstteki not); "zorla sil" önce o ürünü/ markayı içeren siparişleri kaldırıp sonra siliyor. Yalnızca test/hatalı veri temizliği için düşünüldü — gerçek sipariş geçmişini silmek üzere kullanılmamalı. İki tıklamalı onay ile (`ConfirmDeleteButton` , native `confirm()` yerine).
- ~~Sipariş listesinden, detaya girmeden hızlı onay/red/iptal~~ — ✅ Tamamlandı, roadmap dışı bir ek kapsam. Admin sipariş listesindeki her satıra durum makinesinin izin verdiği geçişlere göre ikon

butonlar eklendi (`src/app/admin/orders/order-quick-actions.tsx`): onay tek tıkla (not gerekmiyor), red/iptal iki tıkla — ilk tık küçük bir sebep kutusu açar (sunucu tarafı bu durumlarda notu zorunlu tutuyor), ikinci tık (yeşil onay ikonu) o notla `transitionOrderStatus` 'u çağırır.

- ~~Firma çapında "bayi siparişleri onay gerektirsin" ayarı~~ — ✅ Tamamlandı, roadmap dışı bir ek kapsam. Yeni admin `/admin/settings` sayfası (yalnızca `company_admin` rolü değiştirebiliyor) ve `companies.requires_order_approval` sütunu (varsayılan `true` , mevcut davranışı bozmuyor). Kapalıyken `createOrder` cari/kapıda ödeme siparişlerini `submitted → approved` , sanal POS'ta ödeme başarılı olan siparişleri `paid → approved` durumuna otomatik geçiriyor (`changedBy: null` , sistem notu ile) — durum makinesindeki mevcut geçiş kurallarını genişletmeden, yalnızca kimin tetiklediğini değiştiriyor. Onaylanmış bir sipariş bu ayardan bağımsız olarak yine de iptal edilebiliyor çünkü `approved → cancelled` geçişi zaten her üç akışta da tanımlıydı.
- ~~Red/iptalde not zorunluluğu~~ — ✅ Tamamlandı. Yukarıdaki hızlı işlemlere ek olarak sipariş detay sayfasındaki tam durum geçiş formunda (`status-actions.tsx`) da red/iptal seçilirken not boşsa hem istemcide hem `transitionOrderStatus` sunucu tarafında reddediliyor — tek bir yerden atlanabilecek bir kural olmasın diye iki katmanlı.
- ~~Giriş öncesi firma seçimi (çoklu firma desteği)~~ — ✅ Tamamlandı, roadmap dışı bir ek kapsam. `companies` tablosuna `code` sütunu eklendi (`companies_code_unique`); ana sayfa artık doğrudan giriş formuna değil, bir firma kodu/adı giriş ekranına (`src/app/company-gate.tsx`) açılıyor. Seçilen firma `localStorage` 'a (`hb_company_code`) önbelleklenip `/login?company=...` 'a yönlendiriyor; `src/auth.ts` 'teki `authorize` , kullanıcıyı artık yalnızca e-postayla değil `(email, companyId)` ikilisiyle arıyor — bu, aynı e-postanın farklı firmalarda ayrı hesaplara ait olabilmesini sağlıyor (`src/lib/companies.ts:: resolveCompanyByCodeOrName` , kod büyük/küçük harf duyarsız, isim karşılaştırması da yapıyor). Kayıtlı firma artık geçersizse (silinmiş/ yanlış) `/?companyError=1` ile ana sayfaya dönülüp önbellek temizleniyor. Bu değişiklik henüz yalnızca bir altyapı hazırlığı — bugün hâlâ tek bir firma (Tunam) çalışıyor, ama giriş akışı artık birden fazla firmayı ayrıştırabilecek şekilde kurulu.

Çıkış kriteri — üç ödeme tipiyle de (cari, kapıda ödeme, sanal POS başarılı/başarısız) uçtan uca sipariş oluşturulabiliyor; admin panelde durum geçişleri izlenebiliyor. Tarayıcıda doğrulandı.

Faz 4 — Cari Bakiye & Bağımsız Tahsilat Modülü ✅ Tamamlandı

Hedef: Siparişten kopuk, serbest tutarlı ödeme ve cari hesap defteri.

- ~~Bayi panelinde "Bakiye Öde" ekranı~~ — ✅ Tamamlandı (`src/app/dealer/balance/`). Mevcut checkout'taki mock sanal POS deseniyle aynı: devre dışı sahte kart alanı + "başarısız simüle et" checkbox'ı. Bayi serbest bir tutar girip `mockPosProvider.charge()` ile öder.
- ~~`payments` tablosunda `type = standalone_collection` kaydı~~ — ✅ Tamamlandı. `payBalance` action'ı (`src/app/dealer/balance/actions.ts`) `orderId: null` ile bu tipte bir satır yazıyor; `ChargeInput.orderId` bu yüzden opsiyonel yapıldı (`src/lib/payments/provider.ts`) — bağımsız tahsilatın bağlı olduğu bir sipariş yok.
- ~~`ledger_entries` defteri ve bakiye hesaplama~~ — ✅ Tamamlandı. Yeni tablo (`src/db/schema.ts`) + tek merkezi yazıcı `postLedgerEntry` (`src/lib/ledger.ts`) — her çağrı bir defter satırı yazıp `dealers.currentBalance` önbelleğini aynı anda günceller. Üç yazım noktası: cari sipariş oluşturulduğunda otomatik borç (`src/app/dealer/actions.ts::createOrder`), kapıda ödemesi tahsil edilemeyen siparişin (`delivered_unpaid`) cariye borç düşmesi

(`src/app/admin/orders/actions.ts::transitionOrderStatus`), ve bağımsız bakiye ödemesinin alacak kaydı. Neon HTTP sürücüsünün gerçek transaction desteklememesi nedeniyle adımlar sırayla yazılıyor (Faz 3'teki `createOrder` yorumuyla aynı ödün). Cari bir sipariş reddedilir/iptal edilirse (`transitionOrderStatus`) ya da admin tarafından kalıcı silinirse (`deleteOrder`), oluşturulduğunda otomatik yazılan borç bir ters kayıtla (silmede `ledger_entries` 'teki net bakiyeye göre hesaplanan `manual_adjustment`) dengeleniyor — aksi halde hiç gerçekleşmemiş sayılan bir siparişin borcu bayinin carisinde kalıcı olarak asılı kalırdı.

- ~~Admin tarafında cari hesap ekstresi görünümü~~ — ✅ Tamamlandı (`src/app/admin/dealers/[id]/ledger/page.tsx`) — bayi bazlı borç/alacak dökümü, her satırda o anki bakiye snapshot'ı. Bayi detay sayfasına güncel bakiyeyi gösteren bir "Cari Hesap" özet kartı + ekstre linki eklendi.
- ~~Bayi kataloğunda toplu ürün seçimi + tek işlemle sepete ekleme~~ — ✅ Tamamlandı — Payna Grup Portal karşılaştırmasından gelen ek kapsam, katalog tarafında sipariş verme hızını artırır. Kataloğun sağ üstüne "Tüm miktarları temizle"nin yanına "Miktar Girilenleri Ekle" butonu eklendi (`product-catalog.tsx`): satır satır "Ekle"ye basmadan, miktar yazılmış tüm satırları tek tıkla sepete ekliyor; sepete zaten girmiş bir ürün atlanır (miktarları sessizce toplamak yerine), sipariş katına uymayan miktarlar da atlanır — her iki durumda da hangi ürünlerin neden eklenmediği toast ile bildiriliyor. Bunun için satır bazlı miktar state'i artık `ProductRow` içinde değil üst `ProductCatalog` 'ta tutuluyor (kategori grupları arasında paylaşılabilsin diye); aynı tick içinde art arda çağrılan `addItem` 'ların birbirinin üstüne yazmaması için `useCart` / `useLocalStorageStore` fonksiyonel updater (`prev => ...`) desenine geçirildi. Roadmap'te öngörülen ayrı bir "Ayarlar" sayfasıyla mod seçimine gerek kalmadı (bkz. aşağıdaki not) — tekli ve toplu ekleme her zaman yan yana duruyor.
- ~~"Tekrar sipariş ver"~~ — ✅ Tamamlandı, roadmap dışı bir ek kapsam. "Siparişlerim" listesinde ve sipariş detayında bir buton (`ReorderButton`), geçmiş bir siparişin kalemlerini o markanın sepetine kopyalar. Fiyat/KDV/kat gibi alanlar geçmiş siparişteki değerle değil, ürünün *güncel* değerleriyle kopyalanır (`getReorderItems` , `createOrder` 'ın client fiyatına güvenmeyip sunucuda yeniden okuduğu aynı ilke); artık pasif bir ürün varsa o kalem atlanır ve kullanıcıya adıyla bildirilir (toast). Sepete eklendikten sonra drawer otomatik açılır, dealer miktarları gözden geçirip checkout'a geçer.
- ~~Ürün bazında sipariş katı (adet zorunluluğu)~~ — ✅ Tamamlandı, roadmap dışı bir ek kapsam. `products.order_multiple` (integer, `NOT NULL DEFAULT 1` — 1 = kısıtlama yok); admin ürün ekleme formunda ve `EditProductModal` 'da düzenlenebiliyor, bir "grup" için aynı katın gruptaki her ürüne tek tek uygulanması yeterli (ör. "Kol Börekleri" kategorisindeki ürünlere tek tek 5 atanarak). Bayi kataloğunda ve sepet çekmecesinde miktar bu değerin katı değilse toast ile anlık uyarı gösterilir (input `step` değeri de kata göre ayarlanır); `createOrder` sunucu tarafında aynı kontrolü tekrarlayıp client'a güvenmeden reddeder — mevcut fiyat/KDV doğrulama deseniyle aynı ilke.
- ~~Sepette KDV tutarının ayrı bir satırda gösterilmesi~~ — ✅ Tamamlandı. KDV oranı ürün bazında değişken (`products.vat_rate`); sipariş kalemi oranı sipariş anında snapshot'lanıyor (`order_items.vat_rate` / `vat_amount`) ki ürünün oranı sonradan değişse geçmiş siparişler etkilenmesin. Sepet, checkout ve sipariş detay ekranlarında Ara Toplam/ KDV/Genel Toplam kırılımı gösteriliyor; `orders.total_amount` artık net değil brüt (KDV dahil) tutar — cari limit kontrolü ve mock POS tutarı da buna göre brüt üzerinden çalışıyor.
- ~~Şifremi unuttum akışı~~ — ✅ Tamamlandı (mock-first). `EmailProvider` arayüzü + `MockEmailProvider` (`src/lib/email/`) — ödeme tarafındaki `PaymentProvider` / `MockPosProvider` deseniyle birebir aynı;

gerçek mail gönderilmiyor, link geliştirici konsoluna yazılıyor. Tek kullanımlık, 1 saatlik `password_reset_tokens` tablosu (ham token yerine SHA-256 hash'i saklanıyor). Gerçek bir sağlayıcıya (ör. Resend) geçiş, Faz 6'daki ödeme sağlayıcısı geçişiyle aynı şekilde tek bir implementasyon eklenerek yapılacak — teknoloji yığınına henüz yeni bir servis eklenmedi.

- ~~Sepet ekranının sağdan açılan bir panel (drawer) olarak yeniden tasarlanması~~ — ✅ Tamamlandı. Sepet artık bayi panelinin her ekranından header'daki sepet ikonuyla erişilebilen, marka bazlı gruplu bir drawer (`src/app/dealer/cart-drawer.tsx`) — siparis.paynagrup.com'daki sepet deneyiminin karşılığı, roadmap dışı bir ek kapsamdı.
- ~~Admin ürün sayfasında "Ürünler" bölümünün "Kategoriler" bölümünün üstüne taşınması~~ — ✅ Tamamlandı.
- ~~Ürün tablosunda "Durum" ve "Sil" arasına bir "Düzenle" butonu~~ — ✅ Tamamlandı. Bayi düzenleme modalıyla (`EditDealerModal`) aynı desen: `EditProductModal` (`src/app/admin/products/edit-product-modal.tsx`) — ad, SKU, birim, fiyat, KDV %, kategori ve aktif/pasif tek formdan güncelleniyor. Eskiden ayrı duran satır-içi KDV düzenleyici bu modalın içine taşındığı için kaldırıldı.
- ~~Bayinin kendi siparişini silebilmesi ya da düzenleyebilmesi~~ — ✅ Tamamlandı. Fiziksel silme değil, durum geçişi (iptal) olarak çözüldü — amaç zaten bir bayinin hatalı/yanlış girilmiş bir siparişi iptal edip mevcut "Tekrar sipariş ver" ile tekrar sipariş geçebilmesiydi. Bayi panelinde ("Siparişlerim" listesi ve sipariş detayı) yeni bir "İptal Et" butonu (`src/app/dealer/orders/cancel-order-button.tsx`) — admin'in hızlı red/iptal deseniyle aynı iki adımlı etkileşim (ilk tık sebep kutusu açar, sunucu tarafı notu zorunlu tutar). Yeni `cancelOrder` server action'ı (`src/app/dealer/actions.ts`) sipariş sahipliğini doğruluyor ve durum makinesindeki mevcut `canTransitionOrderStatus` guard'ını (bkz. `src/lib/order-status.ts`) aynen kullanıyor — ek bir kısıtlama yazılmadı, zaten her üç akışta da `cancelled` geçişi yalnızca `submitted` / `approved` 'ta tanımlı olduğundan bayi bir sipariş `preparing` 'e geçtikten sonra artık iptal edemiyor. Cari siparişte oluşturulduğunda otomatik yazılan borç, admin'in `transitionOrderStatus` 'undaki iptal/red bloğuyla birebir aynı desende (`postLedgerEntry` ile alacak kaydı, `sourceType: "order"`) geri alınıyor. **Faz 7 çakışma notu bu tasarımla karşılanmış oldu:** iptal edilebilirlik kontrolü tek bir yerde (`canTransitionOrderStatus`) toplandığı için, ileride eklenecek `orders.erp_synced_at` alanı bu fonksiyona üçüncü bir koşul olarak eklenmesi yeterli olacak, ayrı bir yeniden tasarım gerekmeyecek.
- ~~Bayi ürün kataloğunda hiyerarşik kategori gruplaması~~ — ✅ Tamamlandı, roadmap dışı bir ek kapsam. Kategoriler zaten kendine referans veren `parentId` ile (Faz 2) keyfi derinlikte dallanabiliyordu; eksik olan bunu bayi kataloğunda görsel olarak yansıtmaktı. `src/lib/ category-tree.ts` (`buildCategoryTree`) düz kategori listesini ağaca çevirir, `src/app/dealer/product-catalog.tsx` bunu rekürsif render eder: bir ürün bir alt kategoriye atandığında (ör. "Boşnak Börekleri"), o ürün kataloğunda üst kategorinin ("Börekler") başlığı altında, kendi alt kategori başlığıyla girintili gösterilir; aynı kategorideki ürünler alt alta bir tabloda listelenir. Kategorisiz ürünler "Diğer" başlığı altında toplanır. Aynı gruplama; admin → Ürünler tablosu, sepet çekmecesi, checkout "Sepet Özeti", bayi "Siparişlerim" detayı ve admin sipariş detayı dahil ürün/sipariş kalemi listelenen **her** ekrana uygulandı — paylaşılan `src/components/category-group-list.tsx` (`CategoryGroupList`) bileşeni ağaç kurma + boş kategori dalını atlama mantığını tek yerde tutuyor, her ekran yalnızca kendi satır/tablo görünümünü (`renderItems`) sağlıyor. Sipariş kalemlerinin kategorisi, sipariş anında snapshot'lanmıyor — ürünün *o anki* kategorisi üzerinden gösteriliyor (fiyat/KDV'nin aksine, kategorizasyon parasal bir etkisi olmayan, salt görsel/organizasyonel bir alan olduğundan kasıtlı bir tercih). "Kategori" sütunu artık tekrar etmiyor, yerini gruplama alıyor.

- Bayi kataloğunda "Ekle" satırındaki miktar alanının varsayılanı 1 yerine 0 yapıldı — yanlışlıkla "Ekle"ye basılıp 1 adet eklenmesini önlemek için; miktar 0 iken "Ekle"ye basılırsa "Lütfen bir miktar girin" hatası gösterilir.
- Bayi kataloğuna "Tüm miktarları temizle" butonu eklendi (ürün listesinin sağ üstünde) — henüz sepete eklenmemiş, sadece o an girilmiş miktar alanlarını sıfırlar.
- Header'daki sepet ikonunun yanına "Sepeti boşalt" eklendi — tüm markalardaki sepeti tek seferde boşaltır (`CartContext.clearCart`); sepet boşken tıklanamaz durumda (soluk/pasif).
- ~~Bayi paneli menüsüne bir "Ayarlar" sayfası (sepete ekleme modu seçimi)~~ — farklı şekilde çözüldü, ayrı bir sayfa/ayar gerekmedi. Planlanan Mod 1 (satır satır "Ekle") ve Mod 2 (toplu ekleme), bir tercih anahtarı arkasında seçtirilmek yerine yukarıdaki "Miktar Girilenleri Ekle" özelliğiyle **her zaman birlikte** sunuldu — bayi istediği satırda tek tek "Ekle"ye basabiliyor, istediği zaman birden fazla satıra miktar yazıp tek butonla topluca ekleyebiliyor. Bir ayar ekranı ve kalıcı tercih deposu (localStorage/`users` tablosu) ihtiyacını ortadan kaldırdığı için daha basit bir çözüm.
- ~~Admin ve bayi paneli "Genel Bakış" ekranlarına gerçek KPI ve grafik içeriği~~ — ✅ Tamamlandı, roadmap dışı bir ek kapsam (bkz. Bölüm 5, "Genel Bakış" satırı — önceden yalnızca statik sayaç kartları vardı). Paylaşılan `StatTile` (`src/components/stat-tile.tsx`) ve `BarChart` (`src/components/bar-chart.tsx`) bileşenleri + `src/lib/month-range.ts` (son N ayın kovaları) ve `src/lib/format.ts` (para/sayı biçimlendirme) yardımcıları eklendi. Admin ana sayfası: bayi/ürün sayısı, bu ay sipariş sayısı/cirosu, toplam cari alacak KPI'ları + son 6 ay sipariş sayısı ve ciro trend grafikleri + bu ay kategori bazında en çok sipariş edilen ürünler tablosu. Bayi ana sayfası aynı desende kendi verisiyle: bu ay sipariş sayısı/tutarı, güncel cari bakiye, kalan cari limit + aynı trend grafikleri ve kategori bazlı en çok sipariş verdiği ürünler. İptal/ reddedilen siparişler istatistiklerden hariç tutuluyor (`src/lib/order-status.ts::EXCLUDED_FROM_STATS_STATUSES`) ama teslim edilip tahsil edilemeyen (`delivered_unpaid`) siparişler fiilen gerçekleştiği için dahil tutuluyor. Bayi tarafında ürün kataloğu artık ayrı bir sayfaya taşındı (`/dealer/catalog`), `/dealer` bu genel bakış ekranına ayrıldı; bayi menüsüne "Genel Bakış" ve "Bakiye Öde" eklendi, admin menüsüne "Ayarlar" eklendi.

Çıkış kriteri — bir bayi siparişsiz şekilde mock POS ile ödeme yapıyor ve cari bakiyesinin anında güncellendiğini görüyor.

Faz 5 — Raporlama & Analitik ✅ Tamamlandı

Hedef: Filtrelenebilir, dışa aktarılabilir raporlama motoru.

- ~~Marka / tarih aralığı filtreleriyle dinamik tablo ve grafikler~~ — ✅ Tamamlandı. Ürün bazlı filtre şimdilik kapsam dışı bırakıldı; bayi bazlı filtre de eklenmedi — bayiler raporlarda sadece gruplanmış çıktı (en çok sipariş/ciro yapan bayiler) olarak görünüyor, filtre girdisi değil. Admin genel bakış (`src/app/admin/page.tsx`) ve siparişler (`src/app/admin/orders/`) ekranlarına dönem filtresi (`src/components/period-filter.tsx`, `src/lib/date-range.ts`) ve marka filtresi (`src/components/brand-filter-buttons.tsx`, `src/components/by-brand-toggle.tsx`) eklendi; grafikler seçilen döneme göre saat/gün/hafta/ay bucket'larına bölünüyor (`src/lib/chart-buckets.ts`, `src/components/grouped-bar-chart.tsx`).
- ~~Excel (.xlsx) export~~ — ✅ Tamamlandı, sadece Excel ile — CSV şimdilik kapsam dışı, PDF ise ileride sayısı az birkaç belirli rapor için ayrıca ele alınacak. Yeni `/admin/reports` sayfası (`src/app/admin/reports/`) altında marka bazlı satış raporu ve pasif bayi raporu için birer "Excel'e Aktar"

butonu; indirme, seçili filtrelerle aynı sorguyu tekrar çalıştırıp `exceljs` ile `.xlsx` üreten bir route handler'dan geçiyor (`src/app/admin/reports/export/route.ts`). Rapor sorguları `src/lib/reports.ts` 'te toplandı ki sayfa ve export handler'ı aynı mantığı kullansın.

- ~~Sipariş vermeyen bayi raporu~~ — ✅ Tamamlandı — pasif bayi takibi için Payna Grup Portal karşılaştırmasından gelen ek kapsam. `/admin/reports` sayfasında Bugün / Dün / 1 Hafta / 2 Hafta (en sıkı uç) eşiği seçilebiliyor (`src/lib/inactive-threshold.ts` , `src/app/admin/reports/reports-filters.tsx`); hiç sipariş vermemiş ya da eşik gün sayısından uzun süredir sipariş vermemiş bayiler, hangi marka(lar)a bağlı oldukları ve son sipariş tarihiyle birlikte listeleniyor (`getInactiveDealersReport` , `src/lib/reports.ts`). İptal/red edilen siparişler "sipariş verdi" sayılmıyor — genel bakış istatistikleriyle aynı kural (`EXCLUDED_FROM_STATS_STATUSES`).

Kapsam dışı bırakıldı: büyüyen veri hacmi için özet tablo / materialized view stratejisi — genel bakış grafikleri mevcut hacimde yeterli performans sağlıyor, ileride ihtiyaç olursa ayrı bir işe konu edilecek.

Çıkış kriteri — bir firma yetkilisi son üç ayın marka bazlı satışlarını filtreleyip Excel'e aktarabiliyor.

Faz 6 — Canlı Sanal POS Entegrasyonu & Sertleştirme

Hedef: Mock sağlayıcının gerçek bir bankayla değiştirilmesi ve production sertleştirme.

- `PaymentProvider` arayüzünün gerçek sağlayıcı (Iyzico/Paytr/banka API) implementasyonu
- Webhook/callback doğrulama, 3D Secure akışı
- Yük testi, hata izleme, yedekleme/felaket kurtarma planı

Çıkış kriteri — canlı ortamda gerçek bir kartla uçtan uca ödeme tamamlanabiliyor; mock sağlayıcı hiçbir çağıran kodu değiştirmeden devre dışı kalabiliyor.

Faz 7 — ERP Entegrasyonu

Hedef: Onaylanan siparişlerin şirketin ERP sistemine tek yönlü aktarımı; hangi ERP'ye bağlanılacağı henüz belli olmadığından `PaymentProvider` / `EmailProvider` ile aynı desende bir soyutlama kurulup gerçek bağlayıcı sonradan eklenecek.

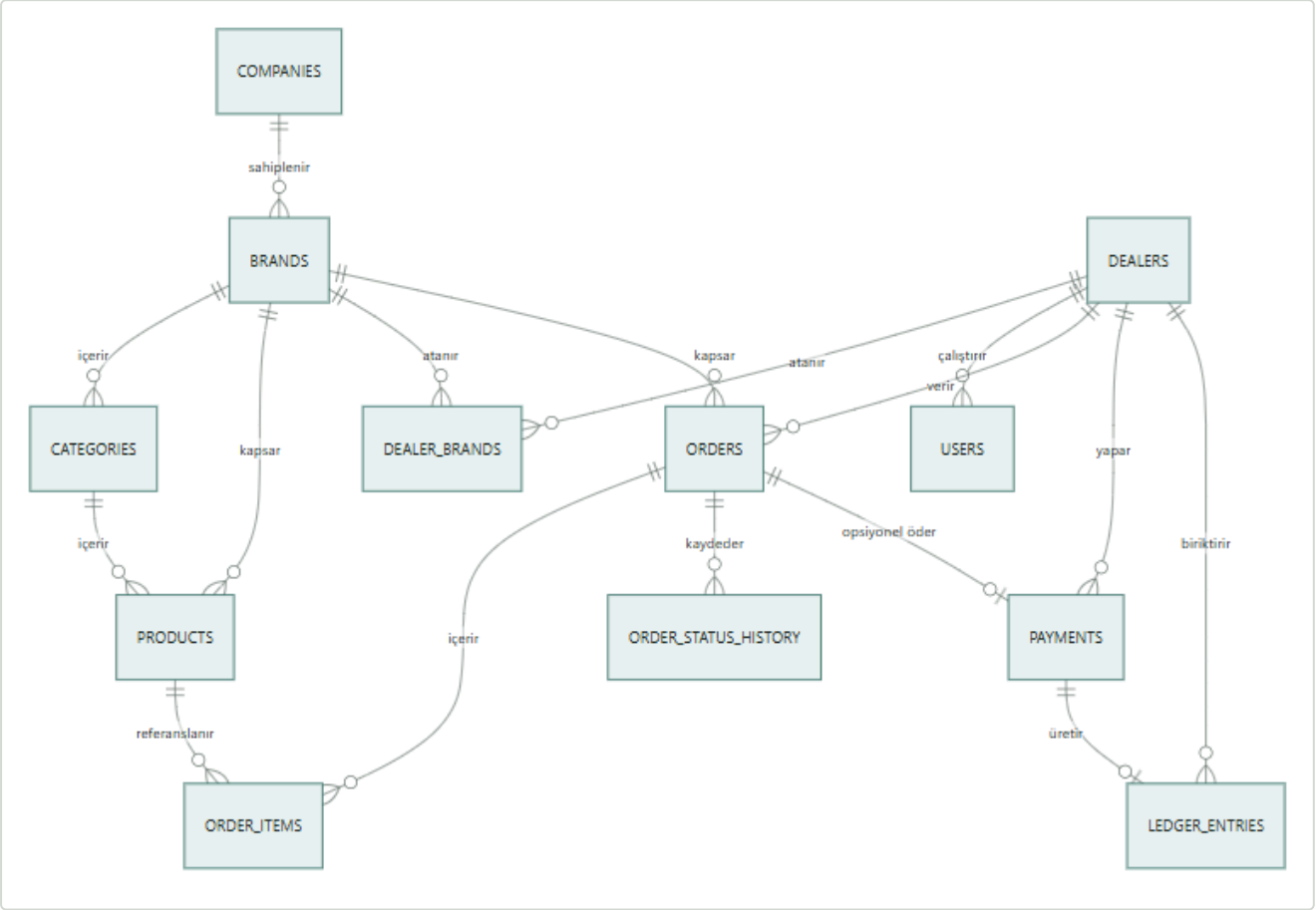
- `ErpSyncProvider` arayüzü + `MockErpSyncProvider` (gerçek ERP seçilene kadar konsola/loglara yazan sahte implementasyon) — `src/lib/payments/` , `src/lib/email/` ile aynı desen
- `orders.erp_synced_at` (nullable timestamp) ve `orders.erp_reference` (ERP tarafındaki kayıt kimliği) alanları
- **Tek yönlü** aktarım (yerel → ERP); ERP'nin siparişi kendi tarafında değiştirip bunu geri yansıtması (webhook/pull + çakışma çözümü) kapsam dışı — hangi ERP'ye bağlanılacağı bilinmeden bu yönde tasarım yapmak erken olur
- Sipariş, `erp_synced_at` dolduktan sonra yerelde salt-okunur olur: ne Faz 4'teki bayi düzenleme/silme özelliği ne admin'in normal silmesi bu siparişe dokunabilir (yalnızca test verisi temizliği amaçlı "zorla sil" dahi ERP'ye aktarılmış gerçek siparişlerde kullanılmamalı — bkz. Faz 4 notu). Düzeltme ihtiyacı iptal + yeni sipariş ile çözülür
- Hangi durumdaki siparişlerin otomatik/manuel aktarılacağı (ör. yalnızca `approved` / `paid` sonrası) ve aktarım tetikleyicisi (durum geçişinde otomatik mi, admin panelden manuel "ERP'ye Aktar" butonuyla mı) uygulama aşamasında netleştirilecek

Çıkış kriteri — onaylanan bir sipariş (mock) ERP'ye aktarılıyor, aktarılan sipariş yerelde düzenleme/silme denemesinde reddediliyor; mock sağlayıcı hiçbir çağıran kodu değiştirmeden gerçek ERP bağlayıcısıyla değiştirilebiliyor.

---

2. Veri Modeli

PostgreSQL / Neon DB uyumlu. Marka bazlı ayrışma her katalog ve stok tablosunda `brand_id` ile; sipariş içi/dışı ödeme ayrımı `payments.order_id`'nin null olup olmamasıyla sağlanıyor.



Kimlik & kiracı yapısı

```
companies
  id            uuid        PK
  name          text
  created_at    timestamptz

brands
  id            uuid        PK
  company_id    uuid        FK -> companies.id
  name          text
  slug          text        UNIQUE
  created_at    timestamptz

dealers
  id                  uuid        PK
  company_id          uuid        FK -> companies.id
  name                text
  tax_no              text
  address             text
  max_balance_limit    numeric(14,2)  NULL   -- NULL = limitsiz; aşılırsa cari kapanır
  current_balance     numeric(14,2)  DEFAULT 0   -- ledger_entries'ten türeyen önbellek
  created_at          timestamptz

dealer_brands
  id            uuid        PK
  dealer_id     uuid        FK -> dealers.id
  brand_id      uuid        FK -> brands.id
  is_active     boolean     DEFAULT true
  UNIQUE(dealer_id, brand_id)

users
  id            uuid        PK
  company_id    uuid        FK -> companies.id
  dealer_id     uuid        FK -> dealers.id   NULL -- null ise firma çalışanı
  email         text        UNIQUE
  role          enum(company_admin, brand_manager, dealer_user)
  password_hash text
  created_at    timestamptz
```

Katalog

```
categories
  id            uuid        PK
  brand_id      uuid        FK -> brands.id
  parent_id     uuid        FK -> categories.id   NULL -- alt kategori için self-reference
  name          text

products
  id            uuid        PK
  brand_id      uuid        FK -> brands.id
  category_id   uuid        FK -> categories.id
  sku           text
  name          text
  unit          text        -- koli / kg / adet
  unit_price    numeric(12,2)
  is_active     boolean     DEFAULT true
  created_at    timestamptz
```

Sipariş

```
orders
  id                  uuid        PK
  dealer_id           uuid        FK -> dealers.id
  brand_id            uuid        FK -> brands.id
  payment_method      enum(cari, cash_on_delivery, virtual_pos)
  status              text        -- bkz. Bölüm 3, durum makinesi
  total_amount        numeric(14,2)
  created_by          uuid        FK -> users.id
  created_at          timestamptz
  updated_at          timestamptz

order_items
  id            uuid        PK
  order_id      uuid        FK -> orders.id
  product_id    uuid        FK -> products.id
  quantity      numeric(10,2)
  unit_price    numeric(12,2)
  line_total    numeric(14,2)

order_status_history
  id            uuid        PK
  order_id      uuid        FK -> orders.id
  from_status   text
  to_status     text
  changed_by    uuid        FK -> users.id   NULL -- sistem tetiklemesiyse boş
  note          text
  created_at    timestamptz
```

Ödeme & cari hareket

```
payments
  id              uuid        PK
  dealer_id       uuid        FK -> dealers.id
  order_id        uuid        FK -> orders.id   NULL -- null ise sipariş dışı bağımsız tahsilat
  type            enum(order_payment, standalone_collection)
  provider        enum(mock, iyzico, paytr)
  amount          numeric(14,2)
  status          enum(pending, success, failed, refunded)
  provider_ref    text        NULL -- banka/POS işlem referansı
  created_at      timestamptz

ledger_entries
  id              uuid        PK
  dealer_id       uuid        FK -> dealers.id
  direction       enum(debit, credit)   -- borç / alacak
  amount          numeric(14,2)
  source_type     enum(order, payment, manual_adjustment)
  source_id       uuid        NULL -- kaynak kayda referans
  balance_after   numeric(14,2)
  created_at      timestamptz
```

> **Neden ayrı bir ledger_entries?** `dealers.current_balance` sık okunan bir önbellektir; gerçek kaynak `ledger_entries` 'teki hareket geçmişidir. Bu ayrım, cari ekstresinin her zaman yeniden hesaplanabilir/denetlenebilir olmasını sağlar.

---

3. Sipariş & Tahsilat Akış Algoritması

Sipariş durum makinesi, checkout anında seçilen `payment_method`'a göre üç ayrı yola dallanır. Her geçiş `order_status_history`'ye yazılır.

Cari (Açık Hesap): draft → submitted → approved → preparing → shipped → delivered → closed. (approved öncesi: cancelled / rejected)

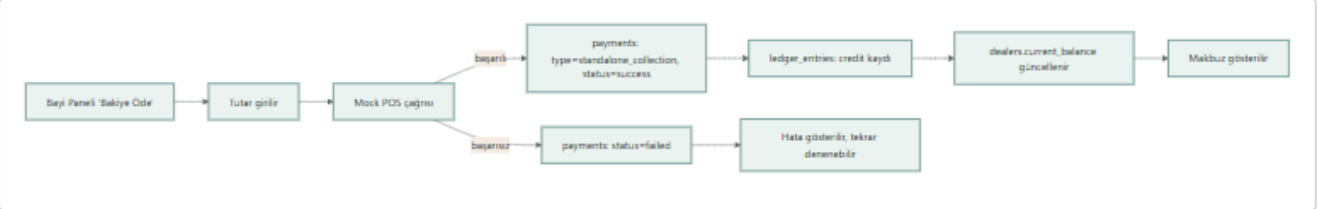
Nakit / Kapıda Ödeme: draft → submitted → approved → preparing → shipped → awaiting_cash_collection → delivered_paid → closed. (tahsil edilemezse: ledger'a borç düşülür, "Bakiye Öde" ile kapanır)

Sanal POS (Mock): draft → awaiting_payment → paid → approved → preparing → shipped → delivered → closed. (POS başarısız: payment_failed → retry / cancelled)

> **Kural:** Sanal POS akışında sipariş, ödeme `success` dönmeden `approved` durumuna geçemez — hazırlık süreci ödemeyle kilitlenir. Cari akışında ödeme adımı yoktur; onay doğrudan hazırlık sürecini tetikler.

Sipariş Dışı Bağımsız Sanal POS Tahsilatı

Bayi, herhangi bir siparişe bağlı olmadan cari bakiyesini kapatmak için serbest tutar girip mock POS üzerinden öder. Bu akış `orders` tablosuna hiç dokunmaz.



---

4. Önerilen Teknoloji Yığını

Tek ekip / hızlı iterasyon senaryosuna göre optimize edildi: Next.js tabanlı tek bir monolit, Neon'un serverless doğasına uygun bir ORM.

KATMAN	ÖNERİ	GEREKÇE
Frontend / Backend	Next.js 15 (App Router) + TypeScript	Server Actions ile admin/bayi panellerini tek repo'da tutar; Vercel + Neon entegrasyonu native.
UI Katmanı	Tailwind CSS + shadcn/ui	Kurumsal tablo/form yoğun ekranlar için hızlı, erişilebilir bileşen tabanı.
Veri getirme	TanStack Query + Server Actions	Sipariş/rapor ekranlarında optimistic update ve cache invalidation.
ORM	Drizzle ORM	Neon'un serverless sürücüsüyle edge-uyumlu, SQL'e yakın; Prisma'ya göre soğuk başlangıç maliyeti düşük.
Veritabanı	Neon Postgres	Zorunlu kısıt. Branch özelliği dev/staging izolasyonu için ayrıca faydalı.
Kimlik doğrulama	Auth.js (credentials) + rol bazlı middleware	Firma admini / marka yöneticisi / bayi kullanıcısı ayrımı route seviyesinde uygulanır.
Önbellekleme	Upstash Redis	Serverless uyumlu; rapor sorgu sonuçları ve oturum verisi için.
Excel / CSV export	exceljs, csv-stringify	Sunucu tarafında akış (streaming) ile büyük raporlarda bellek dostu üretim.
PDF export	@react-pdf/renderer	React bileşenleriyle rapor şablonu; ayrı bir headless tarayıcıya ihtiyaç duymaz.
Grafikler	Recharts	Raporlama ekranındaki filtrelenebilir grafikler için hafif, React-native API.
Ödeme soyutlaması	`PaymentProvider` arayüzü + `MockPosProvider`	Faz 6'da gerçek sağlayıcı (Iyzico/Paytr) sadece yeni bir implementasyon eklenerek takılır.
Hosting	Vercel	Preview deployment'lar + Neon branch'leriyle PR bazlı izole test ortamları.
İzleme	Sentry	Ödeme ve sipariş durum geçişleri gibi kritik akışlarda hata görünürlüğü.
Test	Vitest + Playwright	Playwright özellikle üç farklı checkout akışının regresyona karşı korunması için önemli.

5. B2B Kullanıcı Arayüzü / Ekran Mimarisi

İki ayrı panel, ortak bir bileşen kütüphanesi üzerine kurulu; navigasyon role göre farklılaşır.

Yönetici Paneli

EKRAN	AMAÇ	ÖNE ÇIKAN BİLEŞENLER
Genel Bakış	Aylık sipariş/ciro/cari özeti	KPI kartları, aylık trend grafikleri, kategori bazlı en çok sipariş edilen ürünler
Marka Yönetimi	Alt marka tanımı ve ayarları	Marka CRUD formu, logo/ayar alanları
Bayi Yönetimi	Bayi tanımı, marka ataması, ödeme yöntemi kuralları	Bayi profil formu, marka atama matrisi, cari limit alanı
Ürün & Kategori	Marka bazlı katalog yönetimi	Kategori ağacı, ürün tablosu, toplu fiyat güncelleme
Sipariş Yönetimi	Gelen siparişleri onaylama, durum güncelleme	Durum makinesi zaman çizelgesi, filtrelenebilir sipariş tablosu
Cari Hesap Ekstresi	Bayi bazlı borç/alacak hareket dökümü	Ledger tablosu, bakiye grafiği
Raporlama	Marka/bayi/ürün/tarih filtreli analiz	Filtre çubuğu, dinamik tablo + grafik, Excel/CSV/PDF export butonları

Bayi Paneli

EKRAN	AMAÇ	ÖNE ÇIKAN BİLEŞENLER
Genel Bakış	Sipariş/ciro/cari özet, aylık trend	KPI kartları, sipariş sayısı/tutarı trend grafikleri, kategori bazlı en çok sipariş verilen ürünler
Ürün Kataloğu & Sepet	Erişimi olan marka(lar)ın ürünlerini sepete ekleme	Marka sekmeleri, ürün kartı, sepet çekmecesi
Checkout	Ödeme tipine göre dallanan sipariş tamamlama	Dinamik adım göstergesi, ödeme tipi kartı, mock POS formu (yalnızca Sanal POS seçiliyse)
Siparişlerim	Geçmiş ve aktif siparişlerin takibi	Durum rozeti, zaman çizelgesi görünümü
Cari Bakiye & Serbest Ödeme	Sipariş dışı borç kapatma	Bakiye özeti, serbest tutar girişi, makbuz geçmişi

> **Paylaşılan bileşenler:** rol bazlı kenar çubuğu, durum rozeti (renk = durum ailesine göre; ödeme tipi ayrı bir etiket olarak gösterilir), boş durum illüstrasyonları, tutar alanlarında sağa hizalı sayısal kolonlar.